

HOSPITAL MANAGEMENT SYSTEM (HMS)

Author: Utkarsh Pawade
Live Application:
medicare-hospital.duckdns.org

A modern, full-stack healthcare operations platform built with Spring Boot 3.x, React 18 SPA, and MySQL 8.0

Executive Summary

The **Hospital Management System (HMS)** is a enterprise-grade healthcare platform designed to automate clinical operations and patient care workflows. From public doctor discovery to 30-minute consultation slot scheduling, online Razorpay payments, digital prescription issuance, and pharmacy stock management, HMS modernizes hospital administration under a secure, unified ecosystem.

Platform Highlights & Capabilities

Conflict-Free Scheduling Validates 30-minute time slots (09:00 AM - 05:00 PM) to eliminate physician schedule overlaps.	Online Razorpay Checkout Automated fee processing with HMAC signature verification and email receipts.
Digital Health Prescriptions Doctors issue digital prescriptions tied directly to patient history and diagnoses.	Pharmacy Inventory Management Real-time medicine stock tracking with automatic updates upon prescription issuance.

User Roles & Access Rights

- Patient Portal:** Search specialists, reserve 30-minute consultation slots, complete online payments, view digital prescriptions, and track health history.
- Doctor Portal:** Review upcoming consultation rosters, update availability status, and issue digital prescriptions with custom dosage guidelines.
- Administrator Portal:** Onboard new medical staff, manage hospital departments, oversee pharmacy inventory stock, and monitor analytics.
- Guest Discovery:** Public access to search doctor availability, department fees, and OPD operational hours without logging in.

Technology Architecture & Stack

Component	Technology Stack	Purpose & Details
Frontend SPA	React 18, Vite, Vanilla CSS, Axios, Lucide Icons	Responsive UI, role dashboards, appointment modals
Backend REST	Java 21, Spring Boot 3.x, Spring Security (JWT)	Stateless REST API, JWT auth, RBAC permissions
Persistence	MySQL 8.0, Spring Data JPA, Hibernate ORM	10 Relational tables, indexing, ACID compliance
Integrations	Razorpay Payment Gateway, JavaMailSender (SMTP)	HMAC SHA-256 validation, automated email receipts
DevOps / Cloud	Docker, Docker Compose, AWS EC2 & RDS, Nginx	Containerized deployment, SSL encryption, reverse proxy

Database Architecture (10 Core Tables)

The persistence layer utilizes MySQL 8.0 with 10 structured tables enforcing strict foreign key constraints and indexed lookup paths:

Table Name	Primary Key / Foreign Keys	Description & Attributes
users	userid (PK)	Account credentials, email, password hash, role (PATIENT/DOCTOR/ADMIN)
patients	patient_id (PK), user_id (FK)	Medical history, blood group, emergency contact details
doctor	doctorid (PK), user_id (FK), department_id (FK)	Specialization, qualification, fee, room number, availability
department	departmentId (PK)	Hospital clinical departments (Cardiology, Neurology, etc.)
appointments	appointment_id (PK), patient_id (FK), doctor_id (FK)	Scheduled consultation date, 30-min start/end times, booking status
payments	payment_id (PK), appointment_id (FK), patient_id (FK)	Razorpay order ID, payment ID, HMAC signature, payment status
medicine_master	medicine_id (PK)	Pharmacy stock inventory, trade name, active ingredient, manufacturer
prescription	prescription_id (PK), appointment_id (FK)	Clinical diagnostic notes and doctor instructions
prescription_medicine	prescription_medicine_id (PK), prescription_id (FK)	Prescribed medicine, unit dosage, frequency, course duration
password_reset_token	id (PK), user_id (FK)	Secure UUID tokens for password recovery email workflows

System SLAs & Performance Benchmarks

- **High Request Capacity:** Engineered to serve **≥ 1,000 concurrent requests/sec** using Spring Boot stateless REST services and Hibernate ORM level-2 caching.
- **Low Latency Threshold:** API response times execute under **200 milliseconds** across all endpoints.
- **Data Consistency:** Full ACID compliance across multi-table workflows (Booking → Payment Validation → Stock Deduction).

Code Conventions & Engineering Rules

Category	Casing Pattern	Example	Rule
Classes / Models	PascalCase	Patient, DoctorService	Noun phrases for domain objects
REST Methods	camelCase	bookAppointment()	Verb phrases for service actions
Private Dependencies	_camelCase	_doctorRepo	Underscore prefix for injected fields
Custom Exceptions	PascalCase	ResourceNotFoundException	Mandatory 'Exception' suffix